Widgets, UI, and DevTools

Composing layouts, handling gestures, and debugging

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Week 6

TODAY

What this lab is for

Compose a screen

Build a todo list from layout widgets, then add swipe gestures with Dismissible.

Attach the tools

Launch DevTools, plant three bugs, find them with breakpoints and the inspector.

TIME

2 hours

SUBMIT

Push to `admd-labs`, branch `lab-3`, before Lab 4

SETUP

Where your code goes

1. Open the project from Lab 2 and create a branch: `git checkout -b lab-3`
2. Create `lib/screens/todo_screen.dart`. All three tasks happen there.
3. Run the app once before changing anything. If it does not build, fix that first.
4. Keep the todo app. Task III debugs this exact app with DevTools.

Hint

Hot reload keeps your list contents between edits. Hot restart drops them: use it after changing `initState` or dependencies.

TASK I

The todo screen, element by element

1. A StatefulWidget that holds todos in State.
2. A ListView.builder with one Card per todo.
3. A checkbox in each Card, bound to `done`.
4. A FloatingActionButton that adds a new todo.
5. Extract the Card into a reusable `TodoTile` component.

DONE WHEN

- [ ] Toggling a checkbox rebuilds only that tile.
- [ ] Adding a todo scrolls smoothly.
- [ ] `TodoTile` takes data through constructor.

REFERENCE

Layout widgets you need today

`Column` and `Row`: arrange children vertically or horizontally.

`Expanded`: takes remaining space inside a Row or Column.

`ListView.builder`: lazy list of items; only builds what is visible.

`Card` and `ListTile`: Material design containers.

`Padding`: space around a child; `EdgeInsets` specifies how much.

`SizedBox`: fixed width or height.

`Container`: padding, margin, color, border in one widget.

`Stack`: children drawn on top of each other.

`Scaffold`: AppBar, body, FAB layout.

Hint

Every one takes children, not pixels.

TASK I

Stateless or stateful?

```
class _TodoState extends State<TodoScreen>
    {
  final _items = <Todo>[];
  final _input = TextEditingController();
  void _add() {
    setState(() =>
    _items.add(Todo(_input.text)));
    _input.clear();
  }
  // Dispose the controller in dispose()
}
```

1. StatefulWidget for the screen.
2. StatelessWidget for TodoTile.
3. Pass data through constructor.
4. Call `dispose()` on controllers.

TASK II

The anatomy of one swipeable row

```
Dismissible(
  key: ValueKey(todo.id),
  direction: DismissDirection.endToStart,
  onDismissed: (_) => setState(
      () => _items.removeWhere((t) => t.id
    == todo.id)),
  background: Container(color: Colors.red),
  child: TodoTile(todo: todo),
)
```

1. Wrap each tile in Dismissible.
2. Remove from the list in `onDismissed`.
3. Show a SnackBar for undo.

⚠ Watch out

Index as key causes wrong row to vanish.

TASK III

Launching DevTools

```
# From the IDE (fastest, gives breakpoints)
# VS Code: F5, then Cmd/Ctrl+Shift+P
#          -> "Dart: Open DevTools"
# Android Studio: Run > Debug,
#         then Flutter Inspector

# From terminal (no debugger):
flutter run
# Copy the printed URL into a browser
```

1. Start from the IDE, not bare `flutter run`.
2. Open DevTools Inspector tab.
3. Plant three bugs one at a time.
4. Fix each before the next.

REFERENCE

The DevTools tabs you need today

Inspector
The live widget tree. Select Widget Mode lets you tap a widget on screen. Track widget rebuilds counts how many times each widget rebuilt after an action.

Debugger
Breakpoints, the call stack, and the Variables pane. Conditional breakpoints stop only when a condition is true.

Logging
Everything `debugPrint` writes, plus framework events. Keep it open during debugging.

Performance
Frame timeline and overlay. Only meaningful in a `--profile` build.

Memory
Heap snapshots. Find controllers you forgot to dispose.

Network
Every HTTP request the app makes. You will need this in Lab 4.

TASK III

The three bugs to plant

```
// Bug 1: off-by-one
for (var i = 0; i <= _items.length; i++) {
  done += _items[i].done ? 1 : 0;
}

// Bug 2: no setState
void _toggle(int i) =>
    _items[i].done = !_items[i].done;

// Bug 3: null access
String? _filter;
final visible = _items.where(
    (t) => t.title.contains(_filter!));
```

1. Plant Bug 1, run until it throws.
2. Set breakpoint, watch `i` and length.
3. Fix. Plant Bug 2. Use conditional breakpoint.
4. Plant Bug 3. Fix the null access.

REFERENCE

What rebuilds, and why

`setState` marks one element dirty. Only that subtree rebuilds on the next frame. The framework reuses the element and State when the new widget has the same `runtimeType` and key.

Your list keeps scroll position when one item changes. Keys matter when items can be reordered or removed from the middle.

ONE FRAME

16.7 ms at 60 Hz · 8.3 ms at 120 Hz

Rebuilt is not repainted. Inspector counts rebuilds.

TROUBLESHOOTING

Layout errors and debugging issues

`RenderFlex overflowed by 42 pixels`	Wrap the growing child in Expanded.
`Vertical viewport was given unbounded height`	Wrap the ListView in Expanded.
`A dismissed Dismissible widget is still part of the tree`	Remove the item from the list in `onDismissed`.
`Breakpoints are hollow and never hit`	Start from the IDE (F5), not bare `flutter run`.
`The data changed, the screen did not`	You mutated state outside `setState`. No rebuilds.

Push before you leave.

Branch lab-3 · one commit per task · screenshots of DevTools in the PR description